OOP Coursework Report

DnD Character Creator

Artemij Aldakuškin
Coursework report author

Introduction

This coursework project is a Dungeons & Dragons (DND) helper application with a graphical user interface (GUI). The main goal is to create a digital character sheet creator that supports character creation, ability score generation, health management, history loading, inventory, abilities, and saving/loading character data.

How to run the program?

1. Install dependencies: *pip install pandas openpyxl*
2. Make sure all project files are in the same folder, especially:
- gui.py
- models.py
- score_methods.py
- storage.py
- game_data.xlsx
3. Run the program: *python gui.py*

How to use the program?

1. Enter a character name.
2. Choose a class and (optionally) open the class info window.
3. Choose a race and (optionally) open the race info window.
4. Optionally load a character history from a .txt file.
5. Select an ability score generation method: Standard Array / Dice Roll / Auto By Class.
6. Click CREATE to create a new character.

7. In the character window you can level up / reset level, heal / deal damage, and view stats, modifiers, inventory, abilities, and history.
8. Click SHOW LAST HERO to load the last saved hero from Excel.

Body / Analysis

Functional requirements implementation (DND Helper)

The selected topic is DND Helper, which requires:

- Create/edit characters with name, class, history
- Edit statistics (STR, DEX, CON, INT, WIS, CHA)
- Set up health, inventory, and abilities

Character creation: the character is created from GUI inputs and represented by the Character object.

Ability score generation: the application supports three methods — Standard Array, Dice Roll, and Auto By Class.

Health management: in the character window, HP can be increased/decreased and the hero can level up.

Inventory and abilities: stored as lists in the Character object and saved/loaded through Excel.

Object-Oriented Programming pillars

Encapsulation

Encapsulation is implemented using properties (@property) and setters in the Character class. This allows validation of important fields (e.g., name cannot be empty, level must be in range, HP must be valid).

```
@property
def name(self):
    return self._name

@name.setter
def name(self, value):
    value = str(value).strip()
    if not value:
        raise ValueError("Name can not be empty")
    self._name = value.title()
```

Inheritance

Inheritance is implemented in score generation methods. StandardArrayMethod, DiceRollMethod, and AutoByClassMethod inherit from the base class ScoreGenerationMethod.

```
class ScoreGenerationMethod:
    def generate(self, scores, chosen_class):
        raise NotImplementedError("This method must be overridden")

class StandardArrayMethod(ScoreGenerationMethod):
    def generate(self, scores, chosen_class):
        return [15, 14, 13, 12, 10, 8]
```

Polymorphism

Polymorphism is demonstrated because all score generation strategies share the same interface: generate(). The GUI can call generate() without knowing which exact class is used.

```
score_method = create_score_method(selected_method)
generated_array = score_method.generate(scores, chosen_class)
```

Abstraction

Abstraction is shown by the base class ScoreGenerationMethod, which defines abstract behavior (a method that must be implemented). Concrete classes provide different implementations.

Design pattern: Factory Method

The project uses the Factory Method design pattern in the function create_score_method(). Based on user selection, the factory returns the correct generation object

```
def create_score_method(type_of_method):
    if type_of_method == "Standard Array":
        return StandardArrayMethod()
    elif type_of_method == "Dice Roll":
        return DiceRollMethod()
    elif type_of_method == "Auto By Class":
        return AutoByClassMethod()
    else:
        return None
```

This approach simplifies object creation, keeps GUI logic clean, and makes the program easy to extend.

Composition / Aggregation

The project uses composition/aggregation relationships:

- Character contains an AbilityScores object
- Character contains lists of Item objects (inventory)
- Character contains lists of Ability objects (abilities)
- Character stores connected race and class data

Reading from file & writing to file

The project uses file input/output with:

- game_data.xlsx — stores races, classes, and the last saved hero
- .txt file — optional history text loaded from disk

Saving hero to Excel stores stats, HP, history, inventory, and abilities and replaces the HERO sheet with the latest data.

Testing (unittest)

Core functionality is tested using Python's unittest framework in test_kurs.py.

Tests cover Character validation, HP logic, proficiency bonus, inventory/abilities behavior, AbilityScores functions, and score generation strategies.

To run tests: *python -m unittest test_kurs.py*

Results

- A working GUI application was created for DND character creation and management.
- Three score generation strategies were implemented and integrated using a Factory Method pattern.
- Character data is saved and loaded from Excel, including inventory and abilities.
- Unit tests were written using unittest to verify core program logic.
- A key challenge was keeping data consistent between GUI objects and Excel storage (including handling empty values).

Conclusions

This coursework project successfully implements a DND character helper using OOP principles.

The program demonstrates all four OOP pillars, uses a design pattern, applies composition, supports file input/output, and includes unit testing.

Future development could improve the project by allowing users to edit inventory and abilities directly in the GUI, supporting multiple saved heroes, improving error handling for invalid/missing Excel data, and expanding the race/class database